

# PICC API 使用说明

```
// 只需传递 appIndex, 驱动自动从 PICC_Init() 配置中获取 ID 和 channelId
PICC_SendEvent(PICC_APP_TIMESYNC, 0x02U, data, len, PICC_EVENT_WITHOUT_ACK);
PICC_MethodRequest(PICC_APP_OTA, 0x02U, data, 2U, PICC_METHOD_WITH_RESPONSE);
PICC_MethodResponse(PICC_APP_PWR, 2U, sessionId, 0x00, rspData, 1);
```

## 优势:

- 参数数量从 7-8 个减少到 5-6 个
- 不需要应用层维护 ID 常量, 只需知道自己的 PICC\_APP\_xxx 枚举
- 消除了传错 channelId 或 instanceId 的可能性
- PICC\_Init() 配置保持不变, 完全向后兼容

## 0. 核心通信方向与角色向导 (Directionality & Role Summary)

在深入具体 API 之前, 必须先理清 M核与A核 之间两种通信协议的数据流向。明确了方向, 您就知道什么时候该调用什么接口。

### 1) EVENT 协议 (单向通知 / Fire-and-Forget)

**特性:** 发即忘 (类似 UDP 广播), 发送方无需对方回复处理结果 (无需 Response)。

- **【发送方向: M核 ➡ A核】** ( M -> A )
  - **动作:** M核主动调用 PICC\_SendEvent() 。
- **【接收方向: A核 ➡ M核】** ( A -> M )
  - **动作:** M核通过初始化时注册的 .eventHandler 捕获即时硬中断通知; **或者**在周期任务中主动轮询 PICC\_GetEventData() 读取。

### 2) METHOD 协议 (双向请求 / Request-Response)

**特性:** 一问一答 (类似 RPC), Client 提问发起 Request, Server 作答返回 Response。

- **情形 A: M核是 Client (M核提问, A核作答)**

- **【发起请求：M核 → A核】** ( M -> A )：M核主动调用 `PICC_MethodRequest()` 发送指令。
- **【提取应答：A核 → M核】** ( A -> M )：M核在周期任务中轮询 `PICC_GetResponseData()` 收割 A核的处理结果。

• **情形 B：M核是 Server（A核提问，M核作答）**

- **【提取请求：A核 → M核】** ( A -> M )：M核通过初始化时注册的 `.methodHandler` 捕获 A核的瞬间请求；**或者**在周期任务中轮询 `PICC_GetMethodData()` 拿到 A核发来的指令数据。
- **【反馈应答：M核 → A核】** ( M -> A )：M核在处理完业务后，必须主动调用 `PICC_MethodResponse()` 给 A核擦屁股回包以结束会话。

## 1. 概述

`picc_api.h` 是 M 核应用层与 PICC 驱动层交互的**唯一公共接口头文件**。

应用层只需 `#include "picc_api.h"`，通过 8 个公共 API 完成全部核间通信操作。

## 2. 公共 API 列表

| # | 函数                                  | 方向  | 用途                               |
|---|-------------------------------------|-----|----------------------------------|
| 1 | <code>PICC_Init()</code>            | 初始化 | 注册一个应用模块并配置回调                    |
| 2 | <code>PICC_SendEvent()</code>       | M→A | 发送 Event 通知（简化版：自动使用 Init 时的配置）  |
| 3 | <code>PICC_MethodRequest()</code>   | M→A | 发送 Method 请求（简化版：自动使用 Init 时的配置） |
| 4 | <code>PICC_MethodResponse()</code>  | M→A | 发送 Method 响应（简化版：自动使用 Init 时的配置） |
| 5 | <code>PICC_GetMethodData()</code>   | A→M | 获取 A 核发来的 Method 请求数据及回调结果       |
| 6 | <code>PICC_GetResponseData()</code> | A→M | 获取 A 核返回的 Method 响应数据及回调结果       |
| 7 | <code>PICC_GetEventData()</code>    | A→M | 获取 A 核发来的 Event 通知数据及回调结果        |
| 8 | <code>PICC_GetLinkState()</code>    | 查询  | 查询指定通道的链路连接状态                    |

**重要改进：**从 v2.0 开始，所有发送 API（`SendEvent/MethodRequest/MethodResponse`）都已简化，不再需要手动传递 `providerId`、`consumerId`、`instanceId`、`channelId` 等参数。驱动会自动从 `PICC_Init()` 时注册的配置中获取这些信息。

## 3. PICC\_Init 配置说明

### 3.1 前置条件

在调用 `PICC_Init()` 之前，必须先调用 `PICC_PreOS_Init()`。该函数在 `main()` 函数中、FreeRTOS 调度器启动之前调用，用于初始化 IPCF 驱动和 PICC 基础设施。

```
int main(void)
{
    /* ... 其他初始化 ... */

    PICC_PreOS_Init(); /* 必须在 PICC_Init() 之前调用 */

    vTaskStartScheduler(); /* 启动 FreeRTOS 调度器 */
    /* ... */
}
```

### 3.2 PICC\_AppConfig\_t 结构体

```
typedef struct {
    uint8          localId;          /* 本地 ID (Server 为 ProviderID, Client 为 Cons
    uint8          remoteId;        /* 对端 ID (对端的 ProviderID 或 ConsumerID) */
    PICC_Role_e    role;            /* PICC_ROLE_SERVER 或 PICC_ROLE_CLIENT */
    uint8          channelId;       /* IPCF 通道号 (1 或 2) */
    PICC_MethodCallback_t methodHandler; /* Method 请求回调 (可为 NULL) */
    PICC_EventCallback_t eventHandler; /* Event 通知回调 (可为 NULL) */
} PICC_AppConfig_t;
```

### 3.3 应用索引枚举 (PICC\_AppIndex\_e)

每个应用模块在初始化时需要指定一个唯一的应用索引：

```
typedef enum {
    PICC_APP_PWR      = 0U, /* 电源管理      (ProviderID: 0x01) */
    PICC_APP_OTA      = 1U, /* OTA          (ProviderID: 0x11) */
    PICC_APP_HEALTH   = 2U, /* 健康管理      (ProviderID: 0x21) */
    PICC_APP_COMM     = 3U, /* 通信管理      (ProviderID: 0x31) */
    PICC_APP_STORAGE  = 4U, /* 存储模块      (ProviderID: 0x41) */
    PICC_APP_DIAG     = 5U, /* 诊断模块      (ProviderID: 0x51) */
    PICC_APP_TIMESYNC = 6U, /* 时间同步      (ProviderID: 0x61) */
    PICC_APP_SOA      = 7U, /* SOA 模块      (ProviderID: 0x71) */
    PICC_APP_RSV0     = 8U, /* 预留 0 */
    PICC_APP_RSV1     = 9U, /* 预留 1 */
    PICC_APP_MAX      = 10U /* 最大数量 (数组大小) */
} PICC_AppIndex_e;
```

## 3.2 两个回调字段说明

两个回调字段均为**可选**。传 NULL 表示使用**纯轮询模式**，传函数指针表示使用**即时回调模式**。  
**两种模式可以无缝结合**：邮箱始终存储 A 核原始数据，如果注册了回调，回调产生的结果也会自动存入邮箱。  
**应用层统一通过 PICC\_Get\*Data() 获取全部数据**（既包含 A 核请求/事件原始数据，也包含回调执行后输出的结果 cbResult）。

### methodHandler（Method 请求回调）

| 传值   | 行为                                                                            |
|------|-------------------------------------------------------------------------------|
| NULL | 纯轮询，PICC_GetMethodData() 读取 A 核请求数据。                                          |
| 函数指针 | 请求到达时立刻调用。回调不仅由于构建响应，还可通过 cbResult 输出即时结果，该结果可随后通过 PICC_GetMethodData() 轮询读取。 |

```
typedef uint8 (*PICC_MethodCallback_t)(uint8 consumerId, uint8 methodId,
                                       const uint8 *reqData, uint16 reqLen,
                                       uint8 *rspData, uint16 *rspLen,
                                       uint8 *cbResult, uint16 *cbResultLen);
```

## eventHandler（Event 通知回调）

| 传值   | 行为                                                                |
|------|-------------------------------------------------------------------|
| NULL | 纯轮询，通过 PICC_GetEventData() 读取 A 核 Event 数据。                       |
| 函数指针 | 事件到达时立刻调用。回调内部通常执行轻量、高时效性操作（如采集时间戳）输出到 cbResult 中，并存入邮箱供周期应用轮询读取。 |

```
typedef void (*PICC_EventCallback_t)(uint8 providerId, uint8 eventId,
                                     const uint8 *data, uint16 len,
                                     uint8 *cbResult, uint16 *cbResultLen);
```

### 3.3 统一轮询读取 API（新增 cbResult 极简设计）

```
/* 最后两个参数用于输出本次接收带来的"回调产生的结果（cbResult）"。如果应用不需要或者没注册该类事件的回调 */
sint8 PICC_GetMethodData(appIndex, methodId, data, maxLen, &len, cbResult, &cbLen);
sint8 PICC_GetResponseData(appIndex, methodId, &retCode, data, maxLen, &len, cbResult, &cbLen);
sint8 PICC_GetEventData(appIndex, eventId, data, maxLen, &len, cbResult, &cbLen);
PICC_LinkState_e PICC_GetLinkState(channelId);
```

PICC\_GetLinkState() 是公开的 polling API，用于按通道查询当前链路状态：

- 1. PICC\_LINK\_STATE\_DISCONNECTED
- 2. PICC\_LINK\_STATE\_CONNECTING
- 3. PICC\_LINK\_STATE\_CONNECTED

说明：

- 1. 发送类 API 内部已经自动检查链路状态，未连接时会直接返回失败。
- 2. 因此应用层通常不需要在每次发送前自行检查链路。
- 3. 只有当业务确实需要根据链路状态做自己的流程分支时，才需要调用 PICC\_GetLinkState()。

示例：

```

PICC_LinkState_e linkState;

linkState = PICC_GetLinkState(PWR_CHANNEL_ID);

if (linkState == PICC_LINK_STATE_CONNECTED) {
    /* 链路已建立，可继续执行依赖对端在线的业务逻辑 */
} else if (linkState == PICC_LINK_STATE_CONNECTING) {
    /* 链路正在建立，通常保持等待或进入重试/降级状态 */
} else {
    /* PICC_LINK_STATE_DISCONNECTED */
    /* 链路未建立，可跳过本周期发送或记录本地状态 */
}

```

上面这个示例适用于“业务流程分支控制”。如果只是单纯发送报文，则通常不需要先调用它，例如：

```

sint8 ret;
uint8 payload[1] = { (uint8)PWR_STATE_STANDBY };

ret = PICC_SendEvent(PICC_APP_PWR, PWR_EVENT_STATE_NOTIFY,
                    payload, 1U, PICC_EVENT_WITH_ACK);

if (ret == PICC_E_NOT_CONNECTED) {
    /* 发送接口内部已经完成链路检查，这里按失败处理即可 */
}

```

## 4. 使用示例

**核心概念：**不管应用层底层是否注册了回调函数（执行实时操作），周期主任务始终使用相同的 `PICC_Get*Data()` 进行查询。

这一极简设计意味着应用不再需要自己声明复杂的全局共享变量，也能够轻松得到回调产生的即时计算结果。

### 4.1 场景一：纯轮询（Pwsm 电源管理模型）

回调全部传 `NULL`，直接在周期任务中获取并处理数据。由于不需要回调返回的结果，获取函数中传 `NULL, NULL`。

```

void Pwsm_Init(void)
{
    static const PICC_AppConfig_t cfg = {
        .localId          = PWR_PROVIDER_ID,      /* 0x01 */
        .remoteId         = PWR_CONSUMER_ID,      /* 0x06 */
        .role              = PICC_ROLE_SERVER,
        .channelId         = PWR_CHANNEL_ID,       /* 2 */
        .methodHandler     = NULL, /* 纯轮询模式, 无需回调 */
        .eventHandler      = NULL /* 纯轮询模式, 无需回调 */
    };

    (void)PICC_Init(PICC_APP_PWR, &cfg);
}

void Pwsm_CommEvent(void) /* 10ms 周期任务 */
{
    uint8 buf[8];
    uint16 len;

    /* 【发送通知: M → A】(EVENT 发送单向通知) */
    uint8 payload[1] = { (uint8)PWR_STATE_STANDBY };
    (void)PICC_SendEvent(PICC_APP_PWR, PWR_EVENT_STATE_NOTIFY,
        payload, 1U, PICC_EVENT_WITH_ACK);

    /* 【提取请求: A → M】(METHOD 场景B: M核是 Server 获取请求) */
    /* 没注册回调, 不需要 cbResult, 后两个参数传 NULL, NULL */
    if (PICC_GetMethodData(PICC_APP_PWR, PWR_METHOD_STATE_ACK,
        buf, sizeof(buf), &len,
        NULL, NULL) == PICC_E_OK) {
        if (len >= 2U && buf[0] == PWR_CORE_A) {
            /* 处理收到的 Method Request 的业务逻辑... */

            /* 【反馈应答: M → A】(METHOD 场景B: M核必须回复 Response 形成 RPC 闭环!) */
            /* ⚠ 防呆提示: 因为没注册回调函数, 所以必须手动调用 MethodResponse 发送回包结束本次会话! */
            uint8 rspPayload[1] = { 0x00 }; /* 0x00 表示处理成功 */
            (void)PICC_MethodResponse(PICC_APP_PWR, PWR_METHOD_STATE_ACK,
                0U /* sessionId */, 0x00, rspPayload, 1);
        }
    }
}

```

## 4.2 场景二：链路状态 polling（业务按需查询）

如果某个业务模块确实需要根据链路状态做本地分支控制，可以直接查询 `PICC_GetLinkState()`。

```
void Comm_Main(void)
{
    if (PICC_GetLinkState(COMM_CHANNEL_ID) != PICC_LINK_STATE_CONNECTED) {
        /* 链路未建立时，不发业务报文或进入降级处理 */
        return;
    }

    /* 只有链路已连接时才执行业务收发 */
}
```

这个示例体现的原则是：

1. 链路状态通过 polling API 显式查询。
2. 发送 API 内部已经做链路检查，因此这里只在业务需要分支控制时才调用。
3. 这样可以保持 `PICC_AppConfig_t` 简洁，不为未使用的回调保留配置位。

## 4.3 场景三：Event 回调 + 统一读取（时钟同步时间戳模型）

回调中立刻采集硬件定时器戳写入 `cbResult`，随后周期任务无缝通过 `PICC_GetEventData()` 一并拿到 A 核数据和刚才记录的硬件回调结果（本地时间戳）。





```

/* 有了 A核数据(remoteData) + 回调记录的即时本地时间戳(localTs)，即可开始对齐计算 */
TimeSync_CalculateOffset(localTs, remoteData, remoteLen);

/* 【主动发送通知: M → A】(EVENT 发送单向通知) */
/* 同步计算完成后，如果想立刻反向报喜给 A核，因为是 EVENT，发出去就不管了 */
uint8 syncDoneMsg[1] = { 0x01 };
(void)PICC_SendEvent(PICC_APP_TIMESYNC, TIMESYNC_EVENT_SYNC_DONE,
                    syncDoneMsg, 1, PICC_EVENT_WITHOUT_ACK);
}
}

```

## 4.4 场景四：Method 回调 + 统一读取（OTA 写闪存模型）

```
static uint8 OTA_MethodHandler(uint8 consumerId, uint8 methodId,
                                const uint8 *reqData, uint16 reqLen,
                                uint8 *rspData, uint16 *rspLen,
                                uint8 *cbResult, uint16 *cbResultLen)
{
    if (methodId == OTA_METHOD_FLASH_WRITE) {
        /* [即刻进行擦写操作] */
        sint8 ret = Flash_Write(reqData, reqLen);

        /* 【反馈应答准备：M → A】(METHOD 场景B：自动闭环) */
        /* ⚠️ 高级用法防呆提示：由于您在此注册了 Callback 函数！ */
        /* 驱动底层在拿到下面的 rspData 和 rspLen 赋值后，会【全自动调用 PICC_MethodResponse()】替您
        /* 您在下文的 OTA_Main 手动轮询时，就『绝不能』再去调用一次 MethodResponse 重复发包了。 */
        rspData[0] = (ret == 0) ? 0x00U : 0x01U;
        *rspLen = 1U;

        /* 回调结果输出：把本次写入完毕的字节数留给周期应用任务展示用 */
        cbResult[0] = (uint8)(reqLen >> 8U);
        cbResult[1] = (uint8)(reqLen);
        *cbResultLen = 2U;
        return 0x00;
    }
    return 0x01;
}

void OTA_Init(void)
{
    static const PICC_AppConfig_t cfg = {
        .localId      = OTA_PROVIDER_ID,      /* 0x11 */
        .remoteId     = OTA_CONSUMER_ID,      /* 0x16 */
        .role         = PICC_ROLE_SERVER,
        .channelId     = OTA_CHANNEL_ID,      /* 2 */
        .methodHandler = OTA_MethodHandler,    /* <--- 在这里注册回调！ */
        .eventHandler  = NULL
    };
    (void)PICC_Init(PICC_APP_OTA, &cfg);
}

void OTA_Main(void)
{
    uint8 data[32]; uint16 len;
```

```

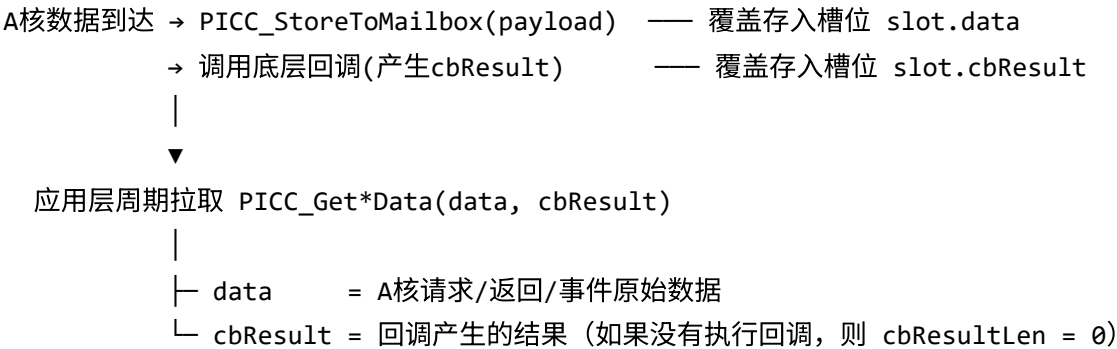
uint8 cbResult[8]; uint16 cbLen;

/* 【主动请求：M → A】(METHOD 场景A: M核主动扮演 Client) */
/* 例如如果 M核 OTA 觉得该下一包了，可以随时发起 MethodRequest 要更新数据 */
// uint8 reqCmd[2] = {0x00, 0x01};
// (void)PICC_MethodRequest(PICC_APP_OTA, OTA_METHOD_REQUEST_DATA,
//                           reqCmd, 2U, PICC_METHOD_WITH_RESPONSE);

/* 【提取请求（已自动回复妥当）：A → M】(METHOD 场景B: 获取通知和回调果实) */
/* 周期任务完全省去写全局变量，在这里直接检查刚才瞬间的回调有没有完成烧写的块 */
if (PICC_GetMethodData(PICC_APP_OTA, OTA_METHOD_FLASH_WRITE,
                      data, sizeof(data), &len,
                      cbResult, &cbLen) == PICC_E_OK) {
    uint16 writtenBytes = ((uint16)cbResult[0] << 8U) | cbResult[1];
    /* 记录 writtenBytes, 更新进度条等... */
}
}

```

# 5. 数据流总结



| 运行时特性  | 注册回调状态 | PICC_Get*Data() 操作结果                     |
|--------|--------|------------------------------------------|
| 纯轮询模式  | NULL   | 读取出 data 为A核原始数据, cbResultLen=0          |
| 异步混合模式 | 函数指针   | 读取出 data 为A核原始数据, 同时 cbResult 为即时回调产生的产物 |

## 6. 兼容性总结

PICC\_Init() 配置接口和 PICC\_Get\*Data() 拉取接口设计为**完全且一致地屏蔽了底层差异**。无论业务对延迟的诉求如何，均可保持主干代码风格统一，极大程度避免在应用层大量维护外部共享标志位，提高高复用性核间通信的使用体验。

## 7. 常见关键疑问解答 (FAQ)

**Q1：我在 PICC\_Init() 中注册了 methodHandler 或者 eventHandler 回调函数，那么在我的周期主函数里，还能/还需要调用 PICC\_GetEventData 或 PICC\_GetMethodData 进行轮询吗？**

**答：绝对可以，并且非常推荐这么做（这就是设计的精髓）！**

底层信箱的机制是\*\*“数据装填”与“回调执行”双管齐下\*\*。

当硬件中断触发、A核的数据瞬间到达时，底层系统会执行两件事：

1. 立马把 A 核传来的\*\*“原始特征数据”\*\*（Payload）塞进信箱槽位。
2. 触发你注册的 Handler 回调。你在回调里可以通过指针写入 cbResult，随后底层也会把你写好的这段 cbResult 追加塞进上述那个同一个信箱槽位！

完成这两步后，中断退出。

**这意味着：即便你注册了 Callback（用于处理一些对时间极度敏感的硬件计算，例如抓拍一条系统时间戳），这批数据依然被完好地封存在信箱中等待回收。**

**随后当你的周期函数（如 10ms task）慢条斯理地执行到时，你依然可以并且应当去调用**

PICC\_GetEventData 或 PICC\_GetMethodData。

这个时候你不仅能原封不动地拿到 A核 发来的 data，还能从出参 cbResult 中完美提取出刚才在中断的回调里顺手帮你算好的“附加结果”。这彻底免去了你在应用层到处定义 extern volatile 全局变量来做跨任务通信的痛苦。

👉 **核心极简示范（以 Event 为例）：**

```

/* 1. 这是你注册在 PICC_Init 中的回调函数（运行在中断级别） */
static void My_EventHandler(uint8 providerId, uint8 eventId,
                           const uint8 *data, uint16 len,
                           uint8 *cbResult, uint16 *cbResultLen)
{
    /* 来报文的瞬间，我只需要做一件对时间极其敏感的事：抓一个当前微秒级时间戳 */
    uint32 hw_timestamp = STM_GetCounter();

    /* 写入到 cbResult 随身包里，底层信箱会替你保管它，供后面主任务提取！ */
    cbResult[0] = (uint8)(hw_timestamp >> 24);
    cbResult[1] = (uint8)(hw_timestamp >> 16);
    cbResult[2] = (uint8)(hw_timestamp >> 8);
    cbResult[3] = (uint8)(hw_timestamp);
    *cbResultLen = 4U;
}

/* 2. 你的 10ms 慢速业务死循环任务（运行在 OS Task 级别） */
void My_PeriodicTask(void)
{
    uint8 rxData[32];    uint16 rxLen;
    uint8 myCbData[8];   uint16 myCbLen;

    /* 重点来了！哪怕你前面用了 Callback，你仍然在这里悠哉地调用 GetEventData */
    if (PICC_GetEventData(PICC_APP_DIAG, 0x01,
                        rxData, sizeof(rxData), &rxLen,
                        myCbData, &myCbLen /* <- 关注这里 */)) == PICC_E_OK)
    {
        /* 一次 API 调用，你拿到了两样东西！ */

        // 1. A核原汁原味发过来的通讯原始报文 (rxData)
        Process_A_Core_Message(rxData, rxLen);

        // 2. 刚才在中断里，你存进 cbResult 的那个微秒级机器时间戳！(myCbData)
        // 跨任务通信连个全局变量都不用定义，是不是爽爆了？
        uint32 fast_timestamp = (myCbData[0] << 24) | (myCbData[1] << 16) | (myCbData[2] << 8)
        Sync_System_Time(fast_timestamp);
    }
}

```

## Q2: PICC\_MethodResponse 到底要在什么场景下使用？是否必须和 PICC\_GetMethodData 成对出现？

答：取决于你初始化时有没有注册回调函数。

场景 1: `methodHandler = NULL` （纯轮询模式）

- 是！必须严格成对使用。
- 流程：使用 `PICC_GetMethodData` 获取请求 → 执行业务 → 必须立刻调用 `PICC_MethodResponse` 回复结果，否则对端会超时死锁。

场景 2: 注册了 `methodHandler` 回调函数（如 OTA 示例）

- 否！绝对不能成对使用，千万不要手动调 `PICC_MethodResponse`。
- 流程：中断触发回调 → 你在这时填好出参 → 底层立刻全自动替你发送 **Response** → 通讯正式结束。如果你在周期任务轮询 `PICC_GetMethodData` 后再去调一遍 `PICC_MethodResponse`，会导致严重重复发包导致系统崩溃。

## Q3: `methodHandler` 和 `eventHandler` 这两个回调函数的执行时间最大不能超过多少？可以调用系统 API 吗？

答：必须极度精简！执行时间严禁超过 50 微秒 (50us)！绝对禁止调用任何带有阻塞性质的 OS API！

由于这两个回调函数是\*\*直接运行在底层 IPCF 硬件 RX 接收中断服务程序 (ISR context) \*\*中的，这赋予了你处理纳秒/微秒级急迫任务（例如对齐时钟时间戳）的权利，但也带来了极高的系统责任：

1. **执行时间极值**：代码必须非常简短，通常只是几条变量赋值、读取一下外设寄存器、或者简单的内存拷贝操作。**整体耗时必须控制在 50us 以内**。千万不能在这里执行耗时的工作（例如：Flash 擦写逻辑、死循环等待设备响应、大量的复杂数学运算等！）。如果遇到这些需要耗时的操作，请把 A 核的指令通过 `cbResult`（或者纯轮询模式）带出来，回抛到外部的 **10ms OS 周期的应用层代码** 里面去慢慢算。
2. **严禁调用阻塞 API（极其致命）**：在中断上下文中，**绝对不允许**调用 `vTaskDelay`、申请带 Wait 时间的 `Semaphore/Mutex` 等一切会引起 OS 上下文挂起的接口。一旦你在里面把 M 核心死锁了或休眠了，整个 FreeRTOS 调度器和其它所有外设的后续中断响应都会瞬间瘫痪，继而直接触发硬件看门狗复位或系统严重假死。